

Week 3: Databases & Storage in Serverless

Database & Storage Engineering Internship Guide

Week 3: Databases & Storage in Serverless

The Complete Theoretical Intern Training & Implementation Guide

Welcome to the Week 3 Databases & Storage in Serverless Intern Guide. Building on the basic serverless computer designs from Week 2, this week explores advanced data persistence and storage integration patterns on Amazon Web Services (AWS). You will learn how to design, secure, and monitor high-performance NoSQL tables, integrate automated storage event notification triggers, and script custom CRUD database transactions.

This guide serves as a self-contained manual. It contains no code snippets (except for step-by-step shell execution commands, file trees, and architecture diagrams), focusing instead on explaining every database layout, event notification trigger, storage class, and SDK operation from first principles. By reading this guide, you will understand exactly what was built during Week 3, how the components interact, and how to deploy and test the architectures step-by-step.

Table of Contents

1. Module 1: NoSQL Databases & Amazon DynamoDB
2. Module 2: S3 Deep-Dive & Event Notifications
3. Module 3: Asynchronous Service Integration
4. Module 4: Programmatic SDK Operations via Boto3
5. Module 5: AWS Security Policies for Databases & Storage
6. Module 6: REST API Gateway Proxy & ANY Method Routing
7. Module 7: Monitoring Asynchronous Storage Flows
8. Module 8: The Notes Management System Architecture
9. Module 9: Terraform Resource Definitions for Database & Storage
10. Module 10: Hands-On Lab & Project Setup Guide
11. Module 11: Week 3 Learning Conclusion

Module 1: NoSQL Databases & Amazon DynamoDB

Traditional relational databases (SQL) organize data into tables with fixed columns and relationships. They require a pre-defined schema and enforce referential integrity. While excellent for complex transactions, SQL databases do not scale horizontally without complex partition systems, and are vulnerable to connection limits when accessed by serverless compute layers.

Amazon DynamoDB is a fully managed, serverless, multi-region NoSQL database. It organizes data into items that contain attributes. It does not enforce a schema (except for primary keys), and scales horizontally by splitting data across multiple physical partitions.

1.1 Data Partitioning & Primary Keys

DynamoDB uses physical storage partitions to host data. To distribute items evenly, it uses primary keys:

- **Partition Key (Hash Key - PK):** The attribute used as an input to an internal hash function. The hash output determines the physical partition where the item will be stored.
- **Sort Key (Range Key - SK):** An optional attribute used to order items physically within the same partition.
- **Composite Primary Key:** Combining a Partition Key and a Sort Key. This allows you to store multiple items under the same Partition Key, sorted by the Sort Key (e.g., Partition Key = `user\_id`, Sort Key = `timestamp`).

1.2 Read/Write Capacity Modes

You can configure how you are billed for database read and write actions:

- **Provisioned Capacity Mode:** You specify the exact number of Read Capacity Units (RCUs) and Write Capacity Units (WCUs) your application requires.
- **RCA definition:** One RCU can perform one strongly consistent read per second, or two eventually consistent reads per second, for items up to 4 KB.
- **WCU definition:** One WCU can perform one write per second for items up to 1 KB.
- **Best for:** Predictable traffic patterns with stable database workloads.

- On-Demand Capacity Mode (Pay-per-Request): DynamoDB automatically scales up and down to accommodate traffic spikes. You do not define RCUs or WCUs; you are billed strictly for the number of read/write requests performed.
- Best for: Serverless applications with unpredictable, bursty traffic.

1.3 DynamoDB Best Practices

- Avoid Scans: A `Scan` operation reads every item in the entire table, consuming massive amounts of capacity. Use `Query` instead, which searches only the partition matching a specific Partition Key.
- Single-Table Design: In NoSQL, you avoid creating multiple tables with relationships. Instead, you store multiple entity types (e.g., Users, Orders, Notes) in a single table, using generic PK and SK names (e.g., `PK = USER#102`, `SK = PROFILE`). This allows you to retrieve all related data in a single query.
- Sparse Indexes: If you create a Global Secondary Index (GSI) on an optional attribute, DynamoDB only writes to that index if the attribute exists in the item. This saves write capacity and storage costs.

Module 2: S3 Deep-Dive & Event Notifications

Amazon S3 is more than a simple storage bucket; it offers advanced lifecycle automation and event triggers.

2.1 S3 Storage Tiers & Cost Optimization

S3 offers different storage classes optimized for different access patterns to reduce costs:

- S3 Standard: Default class. High throughput, low latency, and 99.99% availability. Designed for active, frequently accessed data.
- S3 Standard-IA (Infrequently Accessed): Lower storage cost than Standard, but charging retrieval fees. Suitable for backups or files accessed less than once a month.
- S3 Intelligent-Tiering: Automatically moves data between Standard, IA, and Archive tiers based on access patterns without performance impact, optimizing storage costs automatically.

- S3 Glacier Flexible Retrieval / Deep Archive: Archive storage tiers designed for long-term backups. Retrieval times range from minutes to 12 hours.

2.2 S3 Lifecycle Policies

Lifecycle configurations automate transitions between storage tiers:

- Transition Rules: Automatically move objects to Standard-IA or Glacier after a set number of days (e.g., move files to Glacier after 30 days).
- Expiration Rules: Automatically delete objects after a specified age, helping you clean up old log files or temp assets.

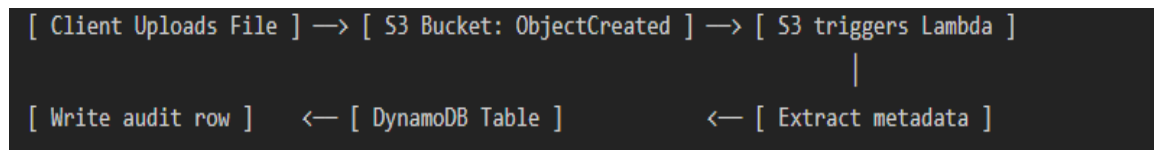
2.3 S3 Event Notifications

S3 can trigger asynchronous workflows in response to bucket actions (like `ObjectCreated` or `ObjectRemoved` events).

- When a file is uploaded, S3 generates a JSON event payload and sends it to a destination target (such as an AWS Lambda function, SQS queue, or SNS topic).
- The event payload contains metadata about the upload, including the S3 bucket name and the object key.

Module 3: Asynchronous Service Integration

In serverless architectures, services are connected asynchronously using event notifications:



- S3 Event Notification: When a file is uploaded (specifically under a path prefix like `incoming/`), S3 triggers a Processor Lambda function.
- Lambda Invocation: The Lambda function receives the S3 event payload. It extracts the bucket name and the object key.
- S3 API Metadata Retrieval: Because the event payload only contains basic metadata, the Lambda function calls the S3 API (`s3\_client.head\_object`) using the bucket name and

key. This retrieves details like the file size (Content-Length) and MIME type (Content-Type).

- **DynamoDB Write:** The Lambda function generates a unique record ID and writes a tracking item containing the S3 file URI, file size, content type, and timestamp to a DynamoDB table.

Module 4: Programmatic SDK Operations via Boto3

Developers interact with AWS resources using SDKs. In Python, this is handled by Boto3.

4.1 Boto3 Client vs. Resource Interfaces

- **boto3.client:** Low-level interface mapping directly to AWS REST APIs. Responses are returned as raw JSON dictionaries.
- **boto3.resource:** High-level, object-oriented abstraction. It wraps AWS APIs in Python classes (e.g., `table = dynamodb.Table("TableName")`), allowing you to execute database operations using clean methods.

4.2 Core SDK Operations

- **put_item:** Inserts a new item or replaces an existing item. If the primary key matches, it overwrites the entire item.
- **get_item:** Retrieves an item using its exact primary key (PK).
- **update_item:** Updates specific attributes of an item without replacing the entire item. It uses update expressions (e.g., `SET feedback = :f`).
- **delete_item:** Deletes an item from the database using its primary key (PK).

Module 5: AWS Security Policies for Databases & Storage

To run Lambda functions securely, you must configure their IAM Execution Roles using the Principle of Least Privilege:

- **S3 Read Permissions:** The S3 Processor Lambda must be granted S3 read permissions (e.g., `s3:GetObject` and `s3:GetObjectVersion`) to retrieve metadata using `head_object`.

- **DynamoDB Write/Read Permissions:** Both the API backend and S3 Processor Lambdas require DynamoDB permissions (such as ``dynamodb:PutItem``, ``dynamodb:GetItem``, and ``dynamodb:DeleteItem``) restricted to the ARN of the target table.
- **S3 Event Permissions:** S3 requires permissions to invoke your Lambda function. This is handled by a resource-based permission rule on the Lambda function that trusts the S3 service principal for the specific bucket ARN.

Module 6: REST API Gateway Proxy & ANY Method Routing

In Week 2, we mapped specific HTTP methods (like GET or POST) to separate API Gateway resources. In advanced production setups, this can be simplified:

- **The ANY Method:** API Gateway allows you to define a wildcard method called ``ANY``. Instead of creating separate API Gateway integrations for GET, POST, PUT, and DELETE, you route all HTTP methods on a resource to a single API Backend Lambda function.
- **Lambda Proxy Integration (AWS_PROXY):** The Lambda function intercepts the raw request, inspects the ``httpMethod`` variable inside the event payload, and routes the request to the appropriate internal function (e.g., if ``POST`` run creation logic, if ``GET`` run query logic).

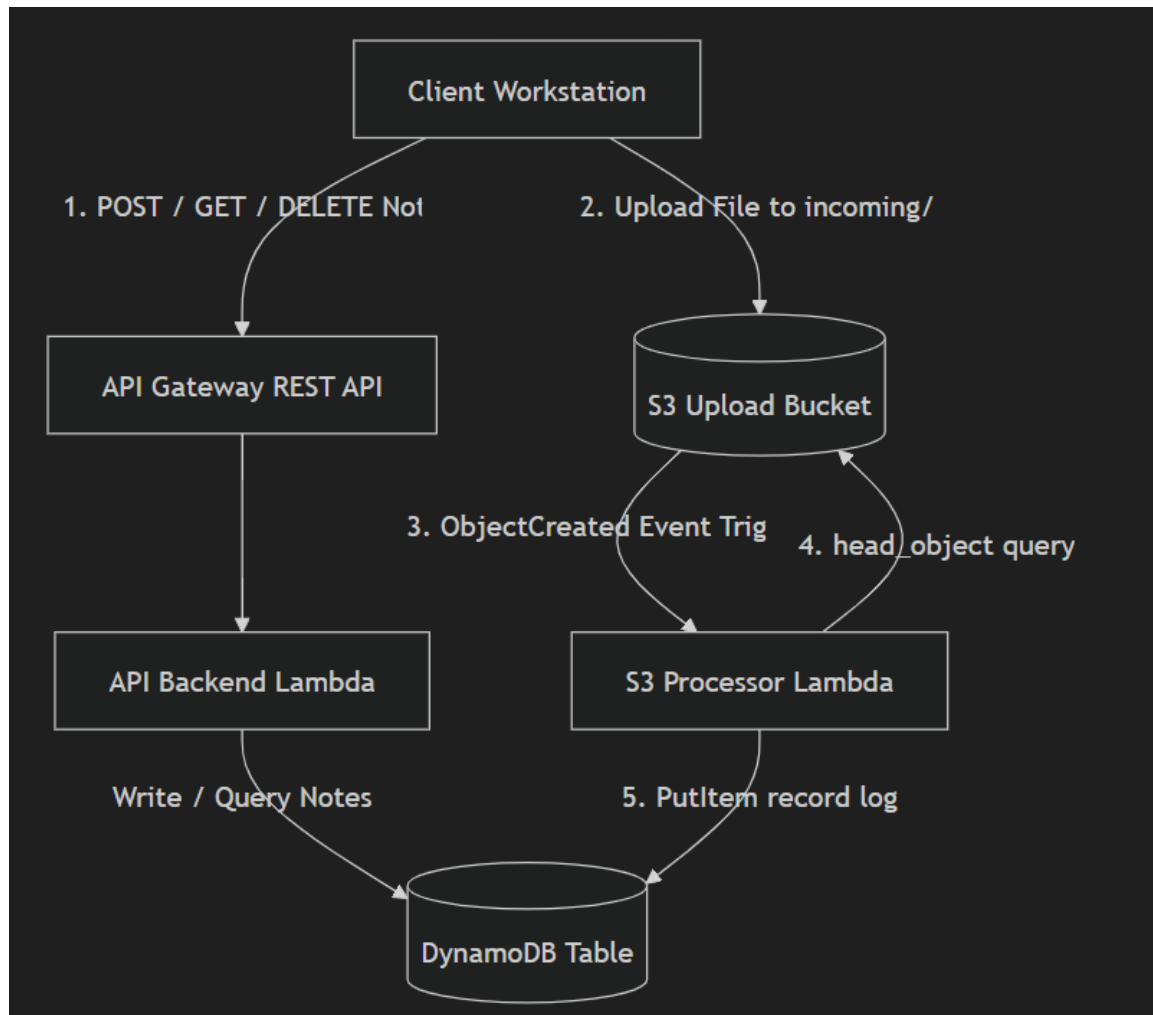
Module 7: Monitoring Asynchronous Storage Flows

When integrating asynchronous event chains, monitoring is critical because failures occur in the background:

- **Exceptions Handling:** In the S3 Processor Lambda, if a ``head_object`` or database write operation fails, the function catches the exception, prints the error stack to CloudWatch Logs, and raises the exception.
- **Event Retries:** Raising the error tells AWS Lambda that the execution failed. If triggered asynchronously by S3, Lambda will automatically retry the execution twice before discarding the event or sending it to a Dead Letter Queue (DLQ).

Module 8: The Notes Management System Architecture

The project for Week 3 is a Notes Management System. It features a CRUD REST API for managing notes, and an automated background tracking system that runs when files are uploaded to S3.



- Synchronous API Path: The client sends HTTP requests via API Gateway. The `api\_backend` Lambda handles:
 - `POST /feedback` \ -> Inserts a feedback/note item in DynamoDB.
 - `GET /feedback?id=<uuid>` \ -> Retrieves a specific item using its partition key.
 - `DELETE /feedback` \ -> Deletes the specified item.

- Asynchronous S3 Path: The client uploads a file to the S3 bucket under the prefix ``incoming/``. This triggers the ``process_upload`` Lambda, which reads the file metadata and writes a log entry into the DynamoDB table.

Module 9: Terraform Resource Definitions for Database & Storage

We provision this architecture using Terraform resources:

- `aws_dynamodb_table`: Creates the DynamoDB table with ``feedback_id`` as the string partition key (``hash_key``).
- `aws_s3_bucket`: Provisions the upload bucket.
- `aws_lambda_function`: Configures both the ``s3_processor`` and ``api_backend`` Lambda functions.
- `aws_lambda_permission`: Grants the S3 service (``s3.amazonaws.com``) permission to invoke the S3 Processor Lambda.
- `aws_s3_bucket_notification`: Links S3 bucket events (``s3:ObjectCreated:`` under prefix ``incoming/``) to trigger the Lambda.
- `aws_api_gateway_rest_api`, `resource`, `method (ANY)`, and `integration`: Maps ``/feedback`` requests to the API Backend Lambda using proxy integration.

Module 10: Hands-On Lab & Project Setup Guide

This module details the directories, execution files, and configurations required to deploy and run the Week 3 databases and storage configurations.

10.1 Project File Directory Structure

Below is the directory tree containing the code resources for this week's assignments:

```
c:\Users\Vaibhav.S.P\Downloads\TringApps Intern
└─ Week 3/
    └─ Dynamo DB/
        ├── main.tf          # Full database, S3 notification & API infra
        ├── variables.tf     # Variables configuration
        ├── outputs.tf       # Outputs declaration
        ├── api_backend.py   # CRUD REST API handler
        ├── process_upload.py # S3 object upload event handler
        ├── crud_operations.py # Local Python SDK diagnostic script
        └─ validation_check.txt # Sample placeholder file for S3 upload
```

10.2 Step-by-Step Implementation Runbook

Follow these steps to deploy and test the components of Week 3:

Task A: Basic Tools & Credentials Setup

Ensure your command terminal is authenticated with your AWS account:

- Open your terminal and navigate to the project directory:

```
cd "Week 3/Dynamo DB"
```

- Verify you have active AWS CLI credentials configured:

```
aws sts get-caller-identity
```

Task B: Run Local Programmatic SDK Diagnostics (Boto3)

To verify local connection capabilities before deploying:

- Install Boto3 on your machine:

```
pip install boto3
```

- Run the diagnostic Python script:

```
python crud_operations.py
```

Expected Output: If a local AWS session is active, the script connects to the target database table, performs a test write, reads the verification data, and prints: ``SUCCESS: Read row verification complete.``

Task C: Full API Backend & S3 Processor Deployment

Now we deploy the complete infrastructure:

- Initialize the Terraform backend:

terraform init

- Validate the configuration files:

terraform validate

- Generate the plan:

terraform plan -out=tfplan

- Deploy the infrastructure to AWS:

terraform apply tfplan

- Verification: Capture the output variables printed in the terminal:

1. ``dynamodb_table_name`` (The database name).
2. ``s3_bucket_name`` (The upload bucket name).
3. ``api_endpoint_url`` (The REST API URL gateway).

Task D: Testing S3 Event Trigger Logging

We test the asynchronous S3 trigger by uploading a file:

- Run the AWS CLI command to copy the ``validation_check.txt`` file to the S3 bucket under the ``incoming/`` prefix:

```
aws s3 cp validation_check.txt s3://vaibhav-photo-uploads-1612/incoming/validation_check.txt
```

- Open the AWS Web Console. Go to DynamoDB \ -> Explore Items.
- Select your table.
- Verification: Check that a new row has been added with:
 1. `username`: `AUTOMATED\_S3\_AGENT`
 2. `feedback`: `File Logged: Name=incoming/validation\_check.txt, Type=text/plain, Weight=100 bytes`
 3. `source\_asset\_uri`: `s3://vaibhav-photo-uploads-1612/incoming/validation\_check.txt`

Task E: Testing REST CRUD API Endpoints

To test the synchronous REST API, use curl in your terminal:

- POST (Create note):

```
curl -X POST -H "Content-Type: application/json" -d '{"username\":"test_user\","feedback\":"Testing notes backend\"}' https://xxxx.execute-api.us-east-1.amazonaws.com/dev/feedback
```

Expected Output: Returns `{"message": "Feedback created successfully", "id": "<uuid>"}`. Capture this UUID.

- GET (Read note):

```
curl "https://xxxx.execute-api.us-east-1.amazonaws.com/dev/feedback?id=<uuid>"
```

Expected Output: Returns the JSON database record containing your username, feedback text, and timestamp.

- DELETE (Delete note):

```
curl -X DELETE -H "Content-Type: application/json" -d '{"id": "<uuid>"}'
https://xxxx.execute-api.us-east-1.amazonaws.com/dev/feedback
```

Expected Output: Returns `{"message": "Feedback record <uuid> deleted successfully"}`.

Task F: Teardown

When you are finished testing, destroy the resources to avoid ongoing AWS charges.

- Run the destroy command:

```
terraform destroy -auto-approve
```

Module 11: Week 3 Learning Conclusion

Throughout this databases and storage systems integration week, we have progressed from basic serverless APIs to event-driven architectures. Here is a summary of the core competencies and learning outcomes achieved:

- **DynamoDB Schema Design:** Mastered composite Partition Keys (Hash Keys) and Sort Keys (Range Keys) to design query-optimized NoSQL tables.
- **Capacity Optimization:** Explored the pricing models and performance trade-offs of On-Demand (pay-per-request) and Provisioned (RCU/WCU) capacity modes.
- **S3 Storage Management:** Learned to optimize storage costs using S3 lifecycle configurations and storage classes.
- **Asynchronous Event notifications:** Configured S3 bucket event notifications to automatically trigger execution functions on file uploads.
- **Boto3 SDK Operations:** Programmed Python scripts to execute CRUD transactions (`put_item`, `get_item`, `delete_item`) on DynamoDB tables.
- **API Gateway ANY Wildcards:** Simplified routing configurations by routing all HTTP methods through a single `ANY` wildcard resource proxy.
- **VPC & Serverless Monitoring:** Inspected execution logs and monitored asynchronous S3 event notifications using CloudWatch log streams.

By combining these skills, you have built a complete, secure infrastructure pipeline, establishing a strong foundation for the rest of your internship!

Appendix A: Glossary of Week 3 Terminology

Acronym / Term	Full Name	Definition / Cloud Role
NoSQL	Non-Relational Database	A database model using non-tabular data structures (e.g., key-value documents) designed for scale.
PK	Partition Key (Hash Key)	The attribute DynamoDB uses as input to a hash function to partition and distribute data physically.
SK	Sort Key (Range Key)	The attribute DynamoDB uses to physically sort items with the same partition key.
RCU	Read Capacity Unit	Unit measuring database read capacity. 1 RCU = 1 strongly consistent read per second for items up to 4 KB.
WCU	Write Capacity Unit	Unit measuring database write capacity. 1 WCU = 1 write per second for items up to 1 KB.
Boto3	AWS SDK for Python	The standard software development kit used by Python developers to programmatically configure and execute AWS APIs.
Client Interface	Boto3 Client	A low-level SDK client mapping 1:1 with raw AWS HTTP service APIs, returning dictionary data structures.

Acronym / Term	Full Name	Definition / Cloud Role
Resource Interface	Boto3 Resource	A high-level, object-oriented SDK abstraction that wraps services as Python classes and resources.
Lifecycle Policy	S3 Lifecycle Configuration	A set of automation rules governing object transitions between storage tiers or automated deletion schedules.

Appendix B: Week 3 Troubleshooting & Debugging

Error Message / Issue	Likely Cause	Step-by-Step Resolution
ResourceNotFoundException on Boto3 script	The table name passed to your Python script does not exist, or your CLI session is targeting the wrong region.	1. Double check the table name spelling in crud_operations.py. 2. Verify your default region configuration in ~/.aws/config or specify region_name='us-east-1' in boto3.resource.
ValidationException: Key schema mismatch	Your SDK request is attempting to write or read an item without providing all the primary keys (PK & SK) matching the table schema.	Check your input dictionary properties. If your table has a sort key, you <i>must</i> specify both the partition key and the sort key exactly as named.
ProvisionedThroughput	Your read or write requests have	1. Implement exponential backoff in your client retry logic.

Error Message / Issue	Likely Cause	Step-by-Step Resolution
tExceededException	exceeded the RCUs/WCUs allocated to your provisioned DynamoDB table.	2. Switch the billing mode to PAY_PER_REQUEST in main.tf to enable On-Demand scaling.
S3 uploads not invoking Lambda	The bucket notification permission trigger is missing or has resource circular dependency issues.	Check main.tf and ensure that aws_lambda_permission.allow_s3 is created and that the bucket notification resource explicitly contains a depends_on = [aws_lambda_permission.allow_s3] block.
AccessDenied on s3:GetObject or PutItem	The execution role assigned to the processor/API Lambda function does not contain policies allowing these actions.	Verify your IAM policies (s3_lambda_s3_read and s3_lambda_db_write attachments) are bound to your Lambda's execution role.